

# ECE 601 – Machine Learning for Engineers

Spring 2026 Course Project – Phase 2: First Draft

University of Massachusetts Amherst

April 14, 2026

## (a) AI Interaction Printout

---

### Original Prompt (Phase 1 — Submitted for Reference)

The following prompt was originally submitted in Phase 1 and used in the initial Phase 2 submission:

Please write a complete Python script that:

1. Loads and preprocesses the images (resize to  $224 \times 224$ , normalize pixel values)
2. Splits the data into training, validation, and test sets
3. Trains a convolutional neural network to classify each image into one of the 7 classes
4. Evaluates performance on the test set using accuracy and macro-averaged F1-score
5. Plots the training and validation loss and accuracy curves

Use PyTorch for the implementation and target a standard GPU (NVIDIA RTX 3060). The script should be runnable end-to-end without requiring any changes. *I have a medical image classification problem. I have a dataset of approximately 10,000 dermatoscopic skin lesion images, each belonging to one of 7 diagnostic categories: melanoma, melanocytic nevi, basal cell carcinoma, actinic keratoses, benign keratosis-like lesions, dermatofibroma, and vascular lesions. The images are in JPEG format and labels are provided in a CSV file with a column called dx. The dataset has significant class imbalance, with the majority of images being melanocytic nevi.*

Please write a complete Python script that:

1. Loads and preprocesses the images (resize to  $224 \times 224$ , normalize pixel values)
2. Splits the data into training, validation, and test sets
3. Trains a convolutional neural network to classify each image into one of the 7 classes
4. Evaluates performance on the test set using accuracy and macro-averaged F1-score
5. Plots the training and validation loss and accuracy curves

Use PyTorch for the implementation and target a standard GPU (NVIDIA RTX 3060). The script should be runnable end-to-end without requiring any changes.

## Revised Prompt (Resubmission — Per Instructor Feedback)

Per instructor feedback, the prompt was revised to remove references to specific implementation steps, model types, and toolboxes, so that it describes only the ML problem and desired outcome. The following revised prompt was submitted verbatim to Claude (Anthropic):

*I have a medical image classification problem. I have a dataset of approximately 10,000 dermatoscopic skin lesion images, each belonging to one of 7 diagnostic categories: melanoma, melanocytic nevi, basal cell carcinoma, actinic keratoses, benign keratosis-like lesions, dermatofibroma, and vascular lesions. The images are in JPEG format and labels are provided in a CSV file with a column called `dx`. The dataset has significant class imbalance, with the majority of images being melanocytic nevi. Please write a complete script that trains a classifier to identify the lesion type from an image, and evaluates its performance on a held-out test set using accuracy and macro-averaged F1-score.*

The AI (Claude, Anthropic) responded to the revised prompt by producing a complete Python script. The key design choices made autonomously by the AI were:

- **Framework:** PyTorch and torchvision
- **Model:** ResNet-18 pretrained on ImageNet, with the final fully connected layer replaced to output 7 classes
- **Preprocessing:** Images resized to  $224 \times 224$  pixels and normalized using ImageNet channel statistics
- **Data split:** 70% training, 15% validation, 15% test using stratified sampling
- **Loss function:** Weighted cross-entropy with inverse-frequency class weights to address class imbalance
- **Optimizer:** Adam with learning rate  $10^{-4}$  and a step LR scheduler decaying by factor 0.1 every 7 epochs
- **Training:** 20 epochs, batch size 32, with data augmentation (random flips, rotation, color jitter) applied to the training set

Minor platform-specific adjustments were made after generation to run the script on Windows (setting `num_workers=0` and removing `pin_memory=True` in the `DataLoader` calls), and to correct the local dataset directory path. No changes were made to the model architecture, training procedure, or evaluation logic.

## (b) Code

---

The full Python script is submitted alongside this document as `Project2Phase2.py`. It was implemented in Python 3.11 using PyTorch and executed on a Windows machine with an NVIDIA RTX 3060 GPU under the `cvpipeline` Anaconda environment.

### (c) Performance Results

---

The script was run to completion over 20 training epochs. The dataset split and per-epoch training log are reproduced below.

#### Dataset Statistics

| Class                 | Label              | Image Count   |
|-----------------------|--------------------|---------------|
| Melanocytic Nevi      | <code>nv</code>    | 6705          |
| Melanoma              | <code>mel</code>   | 1113          |
| Benign Keratosis-like | <code>bkl</code>   | 1099          |
| Basal Cell Carcinoma  | <code>bcc</code>   | 514           |
| Actinic Keratoses     | <code>akiec</code> | 327           |
| Vascular Lesions      | <code>vasc</code>  | 142           |
| Dermatofibroma        | <code>df</code>    | 115           |
| <b>Total</b>          |                    | <b>10,015</b> |

Table 1: Class distribution in the HAM10000 dataset.

The dataset was split into 7,010 training, 1,502 validation, and 1,503 test images using stratified sampling to preserve class proportions across all splits.

## Training Log

| Epoch | Train Loss | Train Acc | Val Loss | Val Acc       |
|-------|------------|-----------|----------|---------------|
| 1     | 1.1895     | 0.5859    | 0.9103   | 0.6578        |
| 2     | 0.8643     | 0.6756    | 0.7537   | 0.6858        |
| 3     | 0.6960     | 0.7107    | 0.8365   | 0.7077        |
| 4     | 0.6388     | 0.7335    | 0.7463   | 0.6997        |
| 5     | 0.5742     | 0.7496    | 0.8190   | 0.6838        |
| 6     | 0.5174     | 0.7689    | 0.7084   | 0.7736        |
| 7     | 0.4545     | 0.7825    | 0.7429   | 0.7383        |
| 8     | 0.3508     | 0.8204    | 0.6019   | 0.7683        |
| 9     | 0.3127     | 0.8308    | 0.5891   | 0.7936        |
| 10    | 0.2954     | 0.8350    | 0.5831   | 0.7776        |
| 11    | 0.2865     | 0.8482    | 0.5784   | 0.7969        |
| 12    | 0.2590     | 0.8484    | 0.5998   | 0.8049        |
| 13    | 0.2549     | 0.8529    | 0.5697   | 0.7989        |
| 14    | 0.2457     | 0.8653    | 0.5501   | 0.8123        |
| 15    | 0.2178     | 0.8700    | 0.5408   | 0.8109        |
| 16    | 0.2282     | 0.8698    | 0.5477   | <b>0.8276</b> |
| 17    | 0.2233     | 0.8698    | 0.5556   | 0.8249        |
| 18    | 0.2310     | 0.8703    | 0.5505   | 0.8229        |
| 19    | 0.2327     | 0.8698    | 0.5728   | 0.8269        |
| 20    | 0.2210     | 0.8750    | 0.5370   | 0.8023        |

Table 2: Per-epoch training and validation loss and accuracy over 20 epochs. Best validation accuracy (0.8276) was achieved at epoch 16.

## Test Set Results

The model checkpoint from epoch 16 (best validation accuracy) was used for final evaluation on the held-out test set:

| Metric            | Value  |
|-------------------|--------|
| Test Accuracy     | 0.8330 |
| Macro F1-Score    | 0.7541 |
| Weighted F1-Score | 0.8400 |

Table 3: Summary of test set performance.

| Class        | Precision | Recall | F1-Score | Support |
|--------------|-----------|--------|----------|---------|
| mel          | 0.53      | 0.70   | 0.60     | 167     |
| nv           | 0.94      | 0.89   | 0.91     | 1006    |
| bcc          | 0.78      | 0.84   | 0.81     | 77      |
| akiec        | 0.62      | 0.78   | 0.69     | 49      |
| bkl          | 0.73      | 0.65   | 0.69     | 165     |
| df           | 0.79      | 0.65   | 0.71     | 17      |
| vasc         | 0.78      | 0.95   | 0.86     | 22      |
| Macro avg    | 0.74      | 0.78   | 0.75     | 1503    |
| Weighted avg | 0.85      | 0.83   | 0.84     | 1503    |

Table 4: Per-class precision, recall, and F1-score on the test set.

## Training Curves

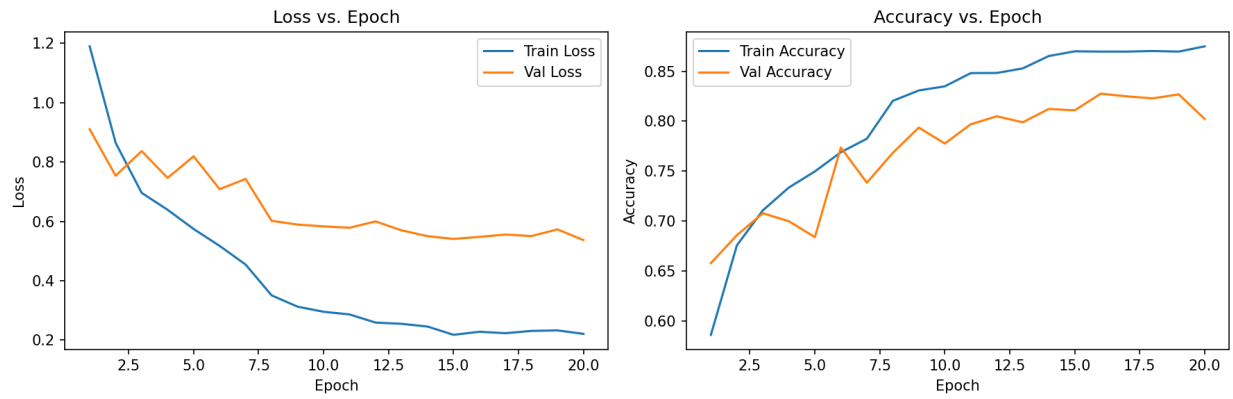


Figure 1: Training and validation loss (left) and accuracy (right) over 20 epochs.

## Confusion Matrix

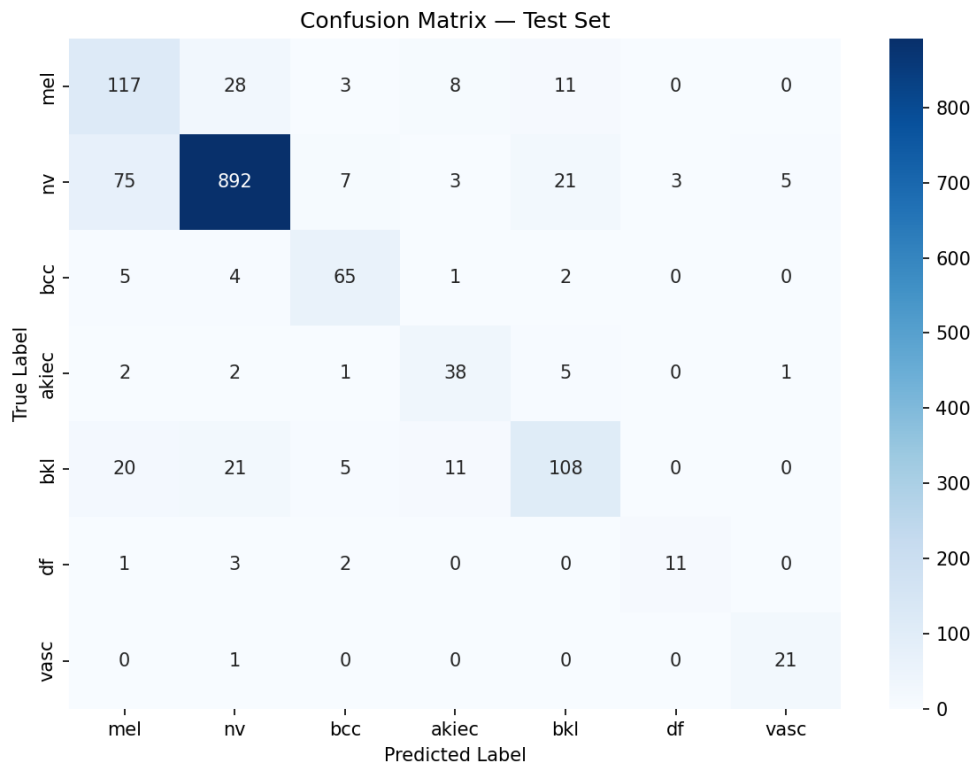


Figure 2: Confusion matrix on the test set. Rows represent true labels; columns represent predicted labels.

## (d) Description of Code and Results

### Model Architecture

The AI selected a **ResNet-18** convolutional neural network pretrained on ImageNet as the base model. ResNet-18 is a residual network consisting of 18 layers organized into four residual block groups, each containing two convolutional layers with skip connections that allow gradients to flow directly through the network during backpropagation. This alleviates the vanishing gradient problem common in deep networks. The final fully connected layer of the pretrained ResNet-18 (originally outputting 1,000 ImageNet classes) was replaced with a new linear layer outputting 7 values, one per diagnostic class. All layers including the pretrained convolutional backbone were fine-tuned end-to-end during training.

### Input Preprocessing

Each input image was resized to  $224 \times 224$  pixels, the standard input resolution for ResNet-18. Pixel values were normalized using the ImageNet channel-wise mean  $[0.485, 0.456, 0.406]$  and standard deviation  $[0.229, 0.224, 0.225]$ , which is appropriate given the pretrained weights. During

training, data augmentation was applied to the training set to improve generalization: random horizontal and vertical flips, random rotation up to  $20^\circ$ , and color jitter (brightness, contrast, and saturation perturbations of  $\pm 0.2$ ). No augmentation was applied at validation or test time.

## Cost Function

The model was trained using **weighted cross-entropy loss**, defined as:

$$\mathcal{L} = - \sum_{c=1}^7 w_c y_c \log(\hat{y}_c)$$

where  $y_c$  is the ground-truth indicator for class  $c$ ,  $\hat{y}_c$  is the predicted softmax probability, and  $w_c$  is a class weight inversely proportional to the class frequency. These weights were computed as  $w_c \propto 1/n_c$ , where  $n_c$  is the number of training samples in class  $c$ , and then normalized so that the weights sum to the number of classes. This weighting scheme penalizes misclassification of minority classes (e.g., `df` with 115 samples) more heavily than the majority class (`nv` with 6,705 samples), directly addressing the significant class imbalance in the HAM10000 dataset.

## Optimization Method

The model was trained using the **Adam optimizer** with an initial learning rate of  $\eta = 10^{-4}$  and default momentum parameters ( $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ). Adam is an adaptive gradient method that maintains per-parameter learning rates by tracking both the first moment (mean) and second moment (uncentered variance) of the gradients, making it well-suited for fine-tuning pretrained networks. A **step learning rate scheduler** was applied, decaying the learning rate by a factor of 0.1 every 7 epochs. This decay schedule is responsible for the visible improvement in both training and validation accuracy beginning around epoch 8, as the smaller step size allows the optimizer to converge more precisely. Training ran for 20 epochs with a batch size of 32.

## Analysis of Results

The model achieved a test accuracy of **83.30%** and a macro-averaged F1-score of **0.7541**. The gap between weighted F1 (0.84) and macro F1 (0.75) reflects the class imbalance: the dominant `nv` class achieves an F1-score of 0.91, while minority classes perform lower. Notably, `mel` (melanoma) achieves only 0.60 F1-score, which is clinically the most important class to detect accurately. The confusion matrix confirms that the most common misclassification is melanoma being predicted as melanocytic nevi (`nv`), a known challenge in the literature due to the visual similarity between the two classes.

The training curves show steady convergence, with train loss decreasing from 1.19 to 0.22 over 20 epochs. The validation loss plateaus around epoch 8, suggesting mild overfitting in the later epochs as the train accuracy continues to climb while validation accuracy stabilizes around 0.82. These observations motivate the improvements to be explored in Phase 3, including stronger regularization, alternative class balancing strategies such as oversampling, and potentially a deeper backbone architecture.